1. Overview

A relational database designed to manage user productivity through structured entities including tasks, meetings, notes, and progress tracking.

The system ensures clear ownership, scalability, and efficient querying for real-world applications.

2. Core Features

- Task management with deadlines and status tracking
- Meeting scheduling with participants
- Notes storage for unstructured data
- Progress tracking for measurable metrics
- User-centric data ownership

3. Core Entities

- **Users** → system identity
- **Tasks** → actionable items
- **Meetings** → scheduled events
- **Meeting Participants** → multi-attendee support
- **Notes** → free text data
- **Progress** → tracked metrics

3.1 Users

Stores user identity and ownership of all resources.

Attribute	Type	Description	Constraints
id	SERIAL	Unique identifier for each user	Primary Key
name	VARCHAR(50)	User’s name	NOT NULL
email	VARCHAR(50)	User’s email address	NOT NULL, UNIQUE
created_at	TIMESTAMP	Creation timestamp	Default: CURRENT_TIMESTAMP

3.2 Tasks Table

The tasks table stores actionable items assigned to users.

Attribute	Type	Description	Constraints
id	SERIAL	Unique task identifier	Primary Key
title	VARCHAR(50)	Short task description	NOT NULL
task_date	DATE	Due date	NOT NULL
task_time	TIME	Due time	NOT NULL
status	ENUM{ 'pending', 'in_progress', 'completed' }	Task state (e.g., pending, completed)	Default: 'pending'
embedding	VECTOR(384)	Semantic embedding	
created_at	TIMESTAMP	Created time	Default: CURRENT_TIMESTAMP
updated_at	TIMESTAMP	Updated time	Default: CURRENT_TIMESTAMP
user_id	INT	Reference to task owner	Foreign Key → users(id), NOT NULL

3.3 Meetings Table

The meetings table stores scheduled events.

Attribute	Type	Description	Constraints
id	SERIAL	Unique meeting identifier	Primary Key
title	VARCHAR(50)	Meeting title	NOT NULL
meeting_date	DATE	Meeting date	NOT NULL
meeting_time	TIME	Meeting time	NOT NULL
status	ENUM{ 'scheduled ', 'completed ', 'in_progress ' }	Meeting state (scheduled, completed, etc.)	Default: 'scheduled'
person	VARCHAR(50	Participant	Optional
location	VARCHAR(50)	Meeting location	Optional
embedding	VECTOR(384)	Semantic embedding	
created_at	TIMESTAMP	Created time	Default: CURRENT_TIMESTAMP
updated_at	TIMESTAMP	Updated time	Default: CURRENT_TIMESTAMP
user_id	INT	Owner of meeting	Foreign Key → users(id), NOT NULL

3.4 Notes Table

The notes table stores unstructured user content.

Attribute	Type	Description	Constraints
id	SERIAL	Unique note identifier	Primary Key
title	VARCHAR(50)	Note title	NOT NULL
description	TEXT	Full note content	NOT NULL
created_at	TIMESTAMP	Created time	Default: CURRENT_TIMESTAMP
updated_at	TIMESTAMP	Updated time	Default: CURRENT_TIMESTAMP
user_id	INT	Owner of note	Foreign Key → users(id), NOT NULL

3.5 note_chunks Table

Attribute	Type	Description	Constraints
id	SERIAL	Unique note identifier	Primary Key
chunk_index	INT	Chunk order	
chunk_text	TEXT	Chunk content	
embedding	VECTOR(384)	Chunk embedding	
created_at	TIMESTAMP	Creation time	Default: CURRENT_TIMESTAMP
note_id	INT	Parent note	Foreign Key → users(id), NOT NULL

3.6 Progress Table

The progress table tracks user metrics over time.

Attribute	Type	Description	Constraints
id	SERIAL	Unique progress identifier	Primary Key
title	VARCHAR(50)	progress title	NOT NULL
status	ENUM{ 'not_started ', 'in_progress ', 'completed' }	progress state (not_started, completed, etc.)	Default: 'not_started'
field	VARCHAR(30)	Category of metric	Optional
value	INT	Recorded value	Optional
embedding	VECTOR(384)	Semantic embedding	
created_at	TIMESTAMP	Created time	Default: CURRENT_TIMESTAMP
updated_at	TIMESTAMP	Updated time	
user_id	INT	Owner of record	Foreign Key → users(id), NOT NULL

4.Relationships, Cardinality, and Participation

1. USERS — TASKS (own)

Cardinality

- One USER can own many TASKS → `1 : N`
- Each TASK belongs to one USER

Participation

- TASKS → **Total participation**
  Every task must belong to a user.
- USERS → **Partial participation**
  A user may have zero tasks.

2. USERS — MEETING (own)

Cardinality

- One USER can own many MEETINGS → `1 : N`
- Each MEETING belongs to one USER

Participation

- MEETING → **Total participation**
- USERS → **Partial participation**

3. USERS — NOTES (own)

Cardinality

- One USER can own many NOTES → `1 : N`
- Each NOTE belongs to one USER

Participation

- NOTES → **Total participation**
- USERS → **Partial participation**

4. USERS — PROGRESS (own)

Cardinality

- One USER can own many PROGRESS records → `1 : N`
- Each PROGRESS record belongs to one USER

Participation

- PROGRESS → **Total participation**
- USERS → **Partial participation**

5. NOTE — NOTE_CHUNKS (RAG)

Cardinality

- One NOTE can have many NOTE_CHUNKS → 1 : N
- Each NOTE_CHUNK record belongs to one NOTE

Participation

- NOTE → Partial participation A note may exist before chunking (raw/unprocessed content)
- NOTE_CHUNKS → Total participation
  A chunk cannot exist without a parent note

Notes:

- All `id` fields are auto-incremented primary keys.
- All `user_id` fields enforce ownership and data isolation.
- Foreign keys use **ON DELETE CASCADE** to maintain integrity.
- Timestamp fields default to current time for automatic tracking

Additional Column to all tables:

- `created_at` → when the row is first inserted Default: CURRENT_TIMESTAMP
- `updated_at` → when the row is last modified Default: CURRENT_TIMESTAMP